

Augura

Our new engine, explained simply.

What we are rebuilding behind Augura, why it is genuinely hard, and how we make sure nothing breaks along the way. No technical background required.

Ideas are easy to claim. Proof is hard to build.

Augura is software that helps health technology companies prove their products actually work. It guides them through designing a credible clinical study: choosing the right patients, the right comparison, the right measurements, then simulating whether the study will convince doctors, insurers and regulators.

The version that exists today is a prototype. It looks finished, it demos beautifully, and it taught us exactly what customers want. But under the surface it is held together the way prototypes always are: **quickly**. This document explains what we are rebuilding, in plain words.

A prototype proves the idea.
A product serves customers.

WHAT STAYS

Everything you can see: the screens, the workflow, the experience. People who saw the demo will notice nothing different.

WHAT CHANGES

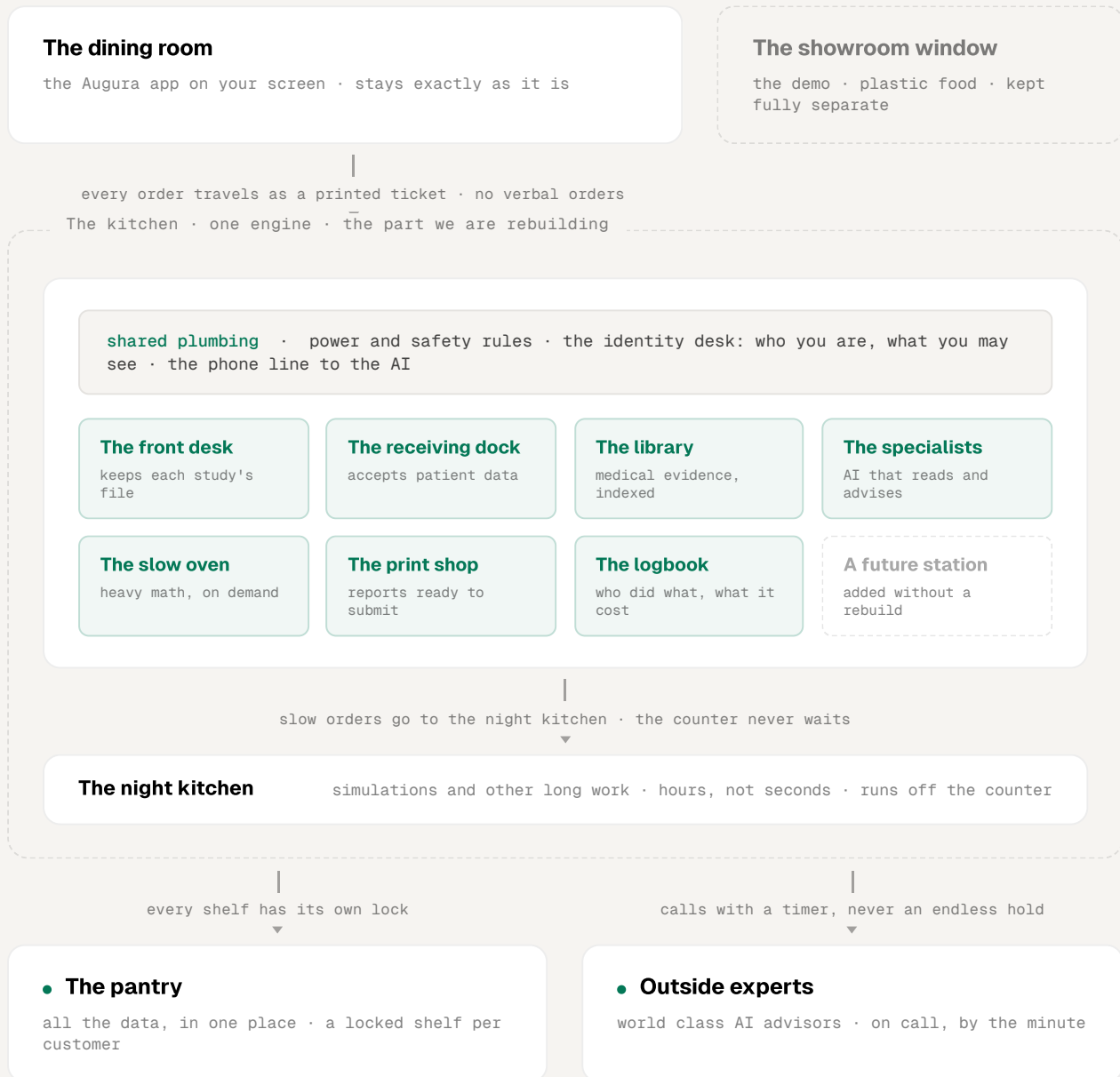
Everything you cannot see: the engine that stores the data, runs the science and talks to the AI is rebuilt on solid ground.

HOW WE STAY SAFE

The old engine keeps running until the new one passes every test. We switch on proof, never on hope.

Think of a restaurant

Software has a part you see and a part you never see. The part you see is the dining room. Everything else is the kitchen, and the kitchen is what we are rebuilding. Here is the whole picture.



One kitchen, seven stations, one pantry. Each station does one job and hands over finished plates; no station ever reaches into another station's drawers. That single rule is what keeps the kitchen calm as it grows, and it is checked by a machine every single day.

Today we run two kitchens

The prototype grew fast, and it grew the way prototypes do: a quick counter up front, a science lab across town, and clever shortcuts holding the two together. Five of those shortcuts are now real problems.

Two kitchens, two languages

The quick counter and the science lab literally speak different programming languages. They cannot share recipes, staff or tools. Every improvement has to be built twice, or it does not happen at all.

A five minute rule at the counter

The current counter must answer every order within five minutes, or the line simply goes dead. Real science takes longer. Today our deepest analyses cannot run there at all: we cook them by hand, off the menu, and paste in the results.

Your file lives on your napkin

The study you build is remembered by your own browser, like an order scribbled on a napkin. Change seats, change devices, or invite a colleague, and the kitchen has no idea who you are. Teamwork is impossible.

Plastic food in the real kitchen

Our demo runs on convincing fake data, like the plastic dishes in a showroom window. Today the plastic and the real food share one kitchen. Sooner or later, someone serves the wrong plate.

Shelves without locks

Each customer's data should sit behind its own lock, so that one client can never see another client's patients. In the prototype, those locks exist on paper but are switched off. That is acceptable for a demo with fake data. It is unthinkable for a product with real data.

None of this shows on screen.
All of it shows at scale.

Six things must stay true at once

No single piece of this rebuild is exotic. The difficulty is that all of the following must hold at the same time, for every customer, on every day of the project. That is what makes engine work slow and careful.

We rebuild while serving

The old engine keeps serving customers during the entire rebuild. Nothing may break in the meantime. The new kitchen opens only when it cooks every dish exactly as well, and we switch in one clean, reversible move.

Walls between customers must be provable

Promising privacy is not enough. We build the locks twice, once in the kitchen and once on the pantry shelves themselves, and an automatic test tries to break in every day. If it ever succeeds, the project stops until it cannot.

A fast lane and a slow lane

A click must answer in a blink; a simulation may take hours. Mixing the two freezes the whole restaurant. The new engine runs two separate lanes and never lets the slow one block the fast one.

The AI needs guardrails

Our AI advisors are brilliant and occasionally wrong. Every answer is checked against a strict form before anyone sees it. If the advisor rambles, we ask once for a correction; if the line drops, you get a clear message, never an endless spinner.

Everything is written down

Every order, answer and failure follows a printed form, and machines check those forms thousands of times a day. This sounds bureaucratic; it is the opposite. What the machine verifies, no human ever has to babysit.

Proof before the switch

We saved every real question the old engine has answered. The new engine must reproduce them all, like a chef passing a blind taste test, dish by dish. Until it passes, the old kitchen stays open and customers notice nothing.

Seven steps, one switch

The rebuild is cut into seven steps. Each step ends with something real that works and is tested. Nothing is left half done, and at no point do we depend on luck.

01	The shell	build the empty kitchen: walls, power, safety inspections, automatic daily checks
02	Identity and files	the identity desk, the locks, and the front desk; the dining room connects for the first time
03	Pantry and library	patient data uploads and the medical library move in, intact, nothing retyped
04	The specialists	the AI advisors move in one by one, the most talkative one last
05	The slow oven	heavy simulations become a button customers press, instead of a phone call to us
06	Print shop and logbook	submission ready reports, plus a clean record of who did what and what it cost
07	The taste test	old and new compared dish by dish; only when every plate matches do we switch

We do not switch on a date.

We switch on proof.

Every saved real case must come out identical from the new engine, end to end, before the old one retires. If a single dish tastes different, we fix it first. Customers never carry the risk.

What you will notice during the rebuild: nothing. What you will notice after: studies that save themselves, colleagues working on the same study together, simulations on demand, and reports generated in minutes instead of days.

Augura

**Simple on the surface.
Solid underneath.**

Questions are welcome. A full technical edition of this document exists for those who want every detail: [Augura-Architecture-Backend.pdf](#)